

Section 20 - Reusing Functionality - Mixins & Custom Composition Functions

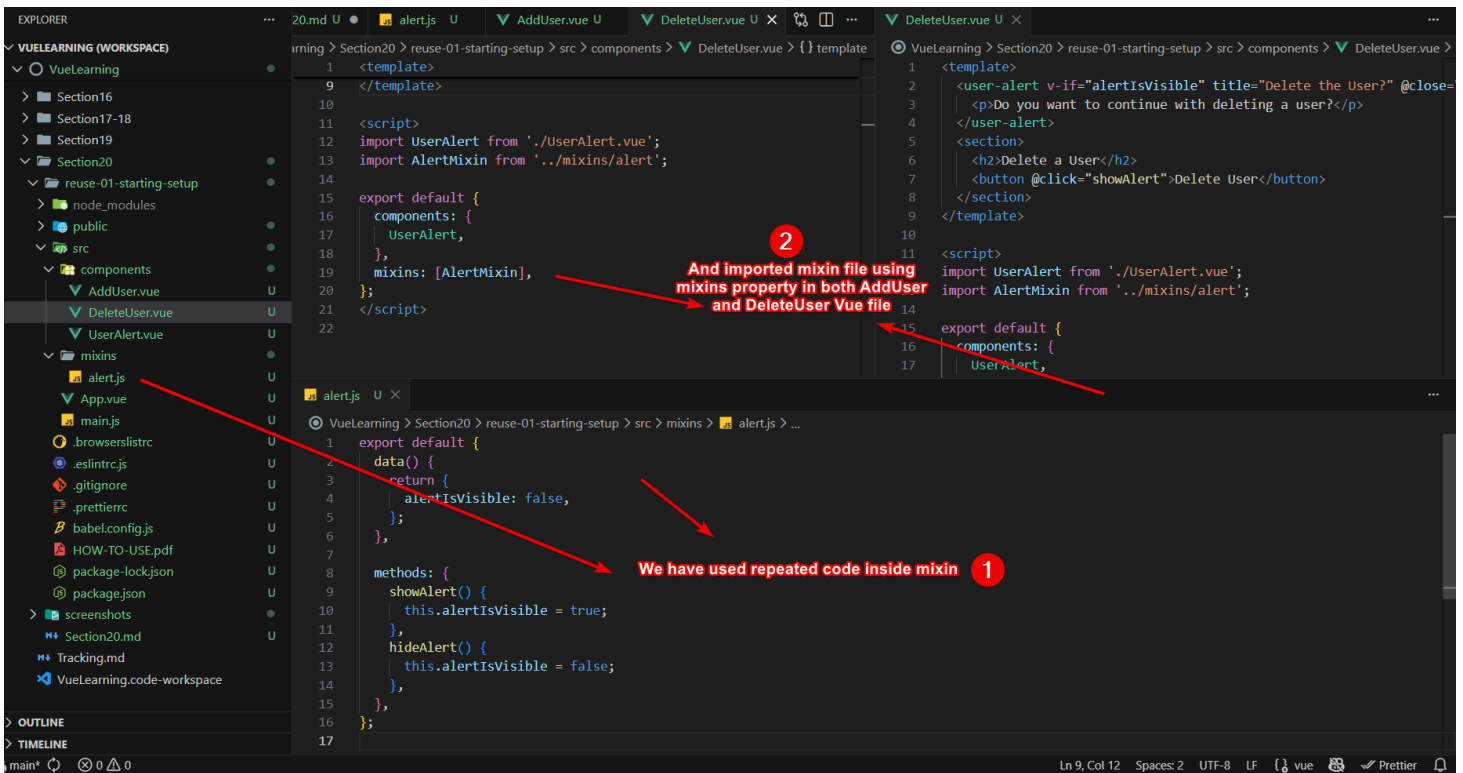
Mixin In Option API

Mixin is used in Option API when we have exactly same `data` , `methods` , `computed properties` , and `watchers` . However, `components` property can't be used.

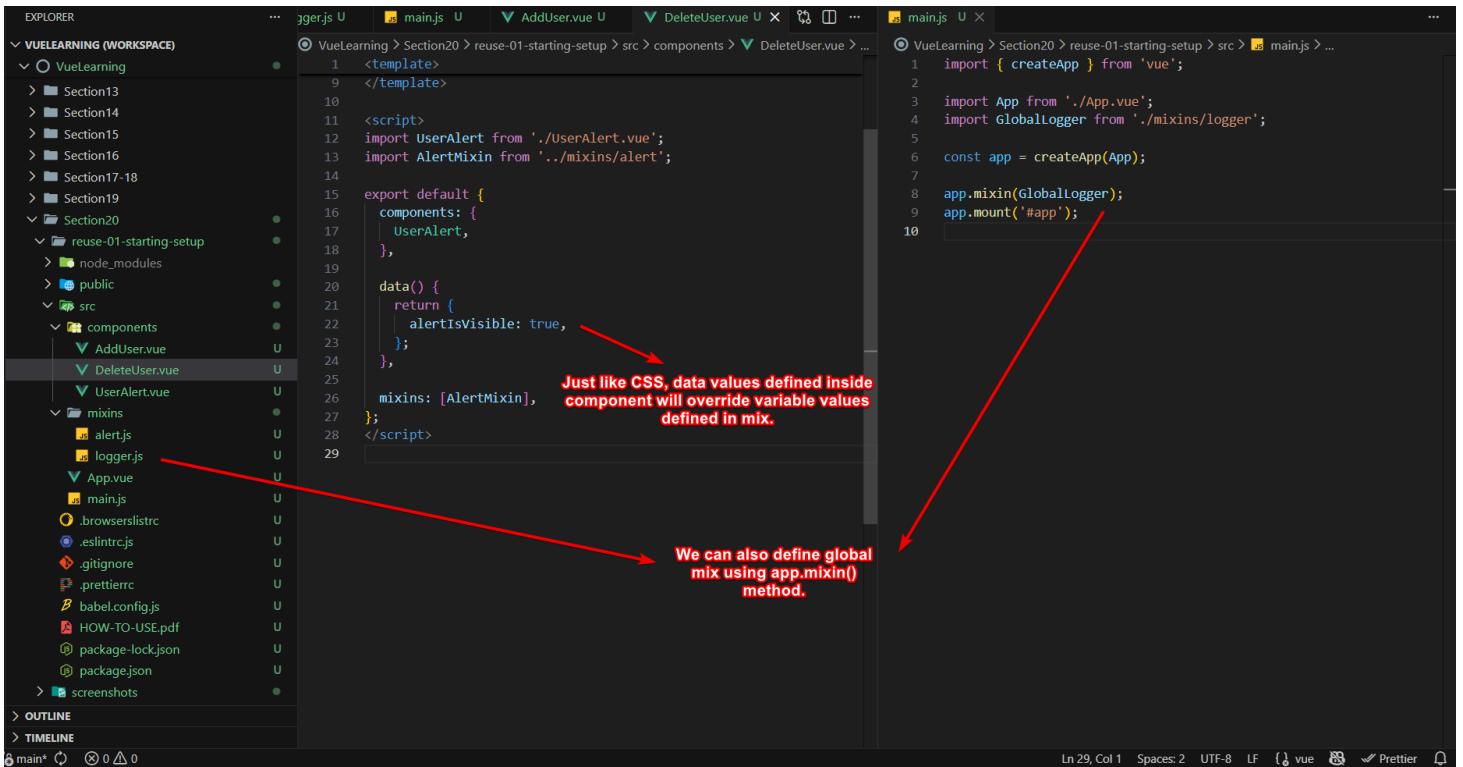
Before Mixin

```
Section20.md U  AddUser.vue U x  DeleteUser.vue U x
VueLearning > Section20 > reuse-01-starting-setup > src > components > AddUser.vue > {} script
1 <template>
2 <user-alert v-if="alertIsVisible" title="Add a User?" @close="hideAlert">
3   <p>Do you want to continue with adding a user?</p>
4 </user-alert>
5 <section>
6   <h2>Add a User</h2>
7   <button @click="showAlert">Add User</button>
8 </section>
9 </template>
10
11 <script>
12 import UserAlert from './UserAlert.vue';
13
14 export default {
15   components: {
16     UserAlert,
17   },
18   data() {
19     return {
20       alertIsVisible: false,
21     };
22   },
23   methods: {
24     showAlert() {
25       this.alertIsVisible = true;
26     },
27     hideAlert() {
28       this.alertIsVisible = false;
29     },
30   },
31 };
32 </script>
Using Mixins we can reuse data, methods, computed properties
etc except components. We can see here data and methods are
exactly same so here we can implement mixin.
DeleteUser.vue > {} template
1 <template>
2 <user-alert v-if="alertIsVisible" title="Delete the User?" @close="hideAlert">
3   <p>Do you want to continue with deleting a user?</p>
4 </user-alert>
5 <section>
6   <h2>Delete a User</h2>
7   <button @click="showAlert">Delete User</button>
8 </section>
9 </template>
10
11 <script>
12 import UserAlert from './UserAlert.vue';
13
14 export default {
15   components: {
16     UserAlert,
17   },
18   data() {
19     return {
20       alertIsVisible: false,
21     };
22   },
23   methods: {
24     showAlert() {
25       this.alertIsVisible = true;
26     },
27     hideAlert() {
28       this.alertIsVisible = false;
29     },
30   },
31 };
32 </script>
```

After Using Mixin



Global Mixin



Custom Hook in Composition API

The screenshot displays a code editor with two files open: `alert.js` and `AddUser.vue`. The `EXPLORER` sidebar on the left shows the project structure, including a `hooks` directory containing `alert.js`.

Annotation 1: Points to the `export default function useAlert(isAlertVisible) {` line in `alert.js`. The text says: "Just like in React, Vue also support creating custom hooks. All you need to do is write the work 'use' as prefix with function name and return whatever values you want."

Annotation 2: Points to the `import { ref } from 'vue';` line in `alert.js`. The text says: "We can pass parameters to custom hook as well."

Annotation 3: Points to the `const [alertIsVisible, showAlert, hideAlert] = useAlert(true);` line in `AddUser.vue`. The text says: "Now we can import the returning values of custom hooks, and pass argument if we want."

Code in `alert.js`:

```
1 import { ref } from 'vue';
2
3 export default function useAlert(isAlertVisible) {
4   const alertIsVisible = ref(isAlertVisible);
5
6   function showAlert() {
7     alertIsVisible.value = true;
8   }
9   function hideAlert() {
10    alertIsVisible.value = false;
11  }
12
13  return [alertIsVisible, showAlert, hideAlert];
14 }
15
```

Code in `AddUser.vue`:

```
11 <script>
12 import UserAlert from './UserAlert.vue';
13 import useAlert from '../hooks/alert';
14
15 export default {
16   components: {
17     UserAlert,
18   },
19   setup() {
20     const [alertIsVisible, showAlert, hideAlert] = useAlert(true);
21
22     return [
23       alertIsVisible,
24       showAlert,
25       hideAlert,
26     ];
27   },
28 }
```